Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Вариант 6

По дисциплине «Основы машинного обучения»

Тема: **«Введение в нейронные сети: построение многослойного перцептрона»**

Выполнил:

Студенты 3 курса

Группы АС-67

Кошелев Б.В.

Проверил:

Крощенко А. А.

Брест 2026

Цель работы: построить, обучить и оценить многослойный **перцептрон (MLP)** для решения задачи классификации

Вариант 6 Диагностика диабета

- Pima Indians Diabetes
- **Задача:** предсказать наличие диабета (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - два скрытых слоя: первый с 12 нейронами, второй с 8 (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** удалите второй скрытый слой (оставив один с 12 нейронами). Сравните recall для класса "диабет" в обеих моделях.

Код программы:

```
import torch
from torch import nn
from torch import optim
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, recall_score

device = (
    torch.accelerator.current_accelerator().type  # type: ignore
    if torch.accelerator.is_available()
    else "cpu"
)
print("Платформа обучения: ", device)

TARGET: str = "Outcome"  # 1 - диабет, 0 - нет диабета
COLUMNS = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age", TARGET,
]
EPOCHS: int = 500


class MLP(nn.Module):
    """
    two_hidden=True : 8 -> 12 (ReLU) -> 8 (ReLU) -> 1
    two_hidden=False: 8 -> 12 (ReLU) -> 1   (эксперимент: второй слой удалён)
    """

    def __init__(self, in_features: int, two_hidden: bool = True) -> None:
        super().__init__()
        layers = [
            nn.Linear(in_features=in_features, out_features=12),
```

```
            nn.ReLU(),
        ]
        if two_hidden:
            layers += [
                nn.Linear(in_features=12, out_features=8),
                nn.ReLU(),
                nn.Linear(in_features=8, out_features=1),
            ]
        else:
            layers += [nn.Linear(in_features=12, out_features=1)]
        # Sigmoid в модель не добавляем: он встроен в nn.BCEWithLogitsLoss,
        # а при предсказании применяется явно (torch.sigmoid / порог logits > 0)
        self.linear_relu_stack = nn.Sequential(*layers)

    def forward(self, x):
        return self.linear_relu_stack(x)


def load_data():
    df: pd.DataFrame = pd.read_csv(
        "pima-indians-diabetes.csv", sep=",", comment="#", header=None,
names=COLUMNS
    )

    # В этих столбцах 0 означает пропуск -> заменяем на медиану
    zero_as_nan = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
    df[zero_as_nan] = df[zero_as_nan].replace(0, np.nan)

    X = df.drop(columns=TARGET)
    y = df[TARGET].values.astype(np.float32)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )

    # Медианы считаем только по обучающей выборке (без утечки данных)
    medians = X_train.median()
    X_train = X_train.fillna(medians)
    X_test = X_test.fillna(medians)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
    X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
    # для BCEWithLogitsLoss метки: float32, форма (N, 1)
    y_train_t = torch.tensor(y_train,
dtype=torch.float32).unsqueeze(1).to(device)
    y_test_np = y_test.astype(int)
```

```
    return X_train_t, X_test_t, y_train_t, y_test_np


def train_and_evaluate(two_hidden: bool, data) -> float:
    X_train, X_test, y_train, y_test = data

    torch.manual_seed(42)
    model = MLP(in_features=X_train.shape[1], two_hidden=two_hidden).to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    print("=" * 45)
    print("\nОбучаемая модель: \n", model)
    print("=" * 45)
    print("\nЦикл обучения модели: ")
    print("\tКоличество эпох: ", EPOCHS)
    print("\tОптимизатор(Adam): ")
    for name, val in optimizer.defaults.items():
        print("\t\t", name, " : ", val)
    print("\tФункция потерь(бинарная): ", criterion)
    print("=" * 45)

    # Цикл обучения
    for _ in range(EPOCHS):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Оценка модели
    print("\nОценка модели нейронной сети: ")
    model.eval()
    with torch.no_grad():
        logits = model(X_test)
        probs = torch.sigmoid(logits)  # Sigmoid на выходе
        predicted = (probs > 0.5).int().squeeze(1).cpu().numpy()

    print(classification_report(
        y_test, predicted, target_names=["нет диабета (0)", "диабет (1)"]
    ))
    return recall_score(y_test, predicted, pos_label=1)


def solution() -> None:
    data = load_data()

    print("\n##### МОДЕЛЬ 1: два скрытых слоя (12 и 8) #####")
```

```
    recall_two = train_and_evaluate(two_hidden=True, data=data)

    print("\n##### МОДЕЛЬ 2: один скрытый слой (12) #####")
    recall_one = train_and_evaluate(two_hidden=False, data=data)

    print("=" * 45)
    print("Сравнение recall для класса 'диабет':")
    print(f"\tДва скрытых слоя (12, 8): {recall_two:.3f}")
    print(f"\tОдин скрытый слой (12):   {recall_one:.3f}")
    print("=" * 45)


if __name__ == "__main__":
    solution()
```

Спецификация вывода:

Платформа обучения: <тип ускорителя обучения>

Обучаемая модель: <слои модели> Цикл

обучения модели:

Количество эпох: <количество эпох обучения>

Оптимизатор: <параметры оптимизатора>

Оценка модели нейронной сети: <оценка по accuracy, precision, recall, f1>

Графические материалы:

```
Цикл обучения модели:
        Количество эпох:  500
        Оптимизатор(Adam):
                lr  :  0.001
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(бинарная):  BCEWithLogitsLoss()
==========================================

Оценка модели нейронной сети:
                precision    recall  f1-score   support

нет диабета (0)       0.79      0.82      0.80       125
     диабет (1)       0.63      0.60      0.62        67

       accuracy                           0.74       192
      macro avg       0.71      0.71      0.71       192
   weighted avg       0.74      0.74      0.74       192


##### МОДЕЛЬ 2: один скрытый слой (12) #####
==========================================

Обучаемая модель:
 MLP(
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=8, out_features=12, bias=True)
    (1): ReLU()
    (2): Linear(in_features=12, out_features=1, bias=True)
  )
)
==========================================
```

```
Цикл обучения модели:
        Количество эпох:  500
        Оптимизатор(Adam):
                lr  :  0.001
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(бинарная):  BCEWithLogitsLoss()
==========================================

Оценка модели нейронной сети:
                precision    recall  f1-score   support

нет диабета (0)       0.76      0.82      0.79       125
     диабет (1)       0.61      0.51      0.55        67

       accuracy                           0.71       192
      macro avg       0.68      0.67      0.67       192
   weighted avg       0.70      0.71      0.71       192

==========================================
Сравнение recall для класса 'диабет':
        Два скрытых слоя (12, 8): 0.597
        Один скрытый слой (12):   0.507
==========================================
```